

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

TASK NO.: 6(PBL Q.no:5)

JavaScript Functions and Classes

Student Name Ch. Venu Gopal Reddy

Roll No. A24126552078

College: Anil Neerukonda Institute of Technology and Sciences (ANITS)

Date: _____

1. TITLE

JavaScript Functions and Classes

2. AIM

To understand the difference between JavaScript functions and classes and to demonstrate how a class can be used to create multiple objects with common properties and methods.

3. OBJECTIVE

- To understand the purpose and syntax of JavaScript functions.
- To understand the concept of classes and objects in JavaScript.
- To create and initialize object properties using a constructor.
- To define and use methods inside a JavaScript class.
- To create multiple objects from a single class using the new keyword.
- To demonstrate that different objects can have different property values while sharing common methods.
- To compare the basic purpose of functions and classes.

4. THEORY

A JavaScript function is a reusable block of code designed to perform a specific task. A function can accept parameters, execute statements and return a value. Functions are useful for avoiding repetition and organizing program logic.

A class in JavaScript is a blueprint for creating objects. A class can contain properties and methods. The constructor() method is used to initialize the properties of an object when the object is created. The new keyword is used to create an object, also called an instance, of a class.

Multiple objects can be created from the same class. These objects have the same set of properties and methods defined by the class, but their property values can be different. In this experiment, a Student class is used to create three student objects with different names, roll numbers and branches.

- JavaScript Functions
- Classes and Objects
- Constructor
- Properties
- Methods
- this keyword
- new keyword
- Multiple Objects

5. ALGORITHM / PROCEDURE

1. Create a JavaScript file named functions_classes.js.

2. Define a normal function named greetStudent() that accepts a student name and returns a greeting.
3. Call the function with different names and display the returned values.
4. Define a Student class using the class keyword.
5. Create a constructor with name, rollNumber and branch parameters.
6. Initialize the object properties using this.name, this.rollNumber and this.branch.
7. Define a displayDetails() method to display the student information.
8. Create three Student objects using the new keyword.
9. Pass different values to the constructor for each object.
10. Call the common displayDetails() method for all three objects.
11. Run the JavaScript program using Node.js.
12. Verify the function output and the details of all created objects.

6. PROGRAM

File: functions_classes.js

Complete JavaScript Program:

```
// Q5: Difference between JavaScript Functions and Classes

// -----
// PART 1: JavaScript Function
// -----

// A function is a reusable block of code.
// It performs a specific task.

function greetStudent(name) {
    return "Hello, " + name + "!";
}

// Calling the function
console.log("FUNCTION EXAMPLE");
console.log(greetStudent("Venu"));
console.log(greetStudent("Rahul"));

// -----
// PART 2: JavaScript Class
// -----

// A class is a blueprint for creating objects.
// It contains properties and methods.

class Student {

    // Constructor initializes the properties
    constructor(name, rollNumber, branch) {
        this.name = name;
        this.rollNumber = rollNumber;
    }
}
```

```

        this.branch = branch;
    }

    // Method to display student details
    displayDetails() {
        console.log("Name:", this.name);
        console.log("Roll Number:", this.rollNumber);
        console.log("Branch:", this.branch);
        console.log("-----");
    }
}

// -----
// PART 3: Creating Multiple Objects
// -----

const student1 = new Student(
    "Venu",
    "A24126552078",
    "CSM"
);

const student2 = new Student(
    "Rahul",
    "A24126552079",
    "CSE"
);

const student3 = new Student(
    "Priya",
    "A24126552080",
    "ECE"
);

// -----
// PART 4: Calling the Common Method
// -----

console.log("\nCLASS OBJECTS");

student1.displayDetails();
student2.displayDetails();
student3.displayDetails();

```

7. CODE EXPLANATION

Element / Statement	Purpose
function greetStudent()	Defines a reusable function that accepts a name and returns a greeting.
class Student	Defines a blueprint for creating Student objects.
constructor()	Initializes the properties of each Student

	object when it is created.
this.name, this.rollNumber, this.branch	Store values in the current object.
displayDetails()	Method that displays the properties of a Student object.
new Student()	Creates a new object (instance) from the Student class.
student1, student2, student3	Three separate objects created from the same Student class.
console.log()	Displays program output in the terminal.

8. FUNCTIONS VS CLASSES

Aspect	Function	Class
Purpose	Performs a specific task	Blueprint for creating objects
Usage	Called directly	Objects are generally created using new
Contents	Statements and logic	Properties, constructor and methods
Objects	Used as reusable code	Used to create multiple objects
Example	greetStudent(name)	class Student { ... }

9. TEST CASES AND EXECUTION DATA

The following test cases can be used to verify the major requirements. Fill the actual result/status after executing the program.

Test Case	Test / Input	Expected Result	Actual Result / Status
TC-01	Call greetStudent("Venu")	Hello, Venu! should be displayed	Passed
TC-02	Call greetStudent("Rahul")	Hello, Rahul! should be displayed	Passed
TC-03	Create student1 using Student class	Venu, A24126552078 and CSM should be displayed	Passed
TC-04	Create student2 using Student class	Rahul, A24126552079 and CSE should be displayed	Passed
TC-05	Create student3 using Student class	Priya, A24126552080 and ECE should be displayed	Passed
TC-06	Call displayDetails() for all objects	Details of all three objects should be displayed	Passed
TC-07	Run complete program	Function output and class object output should appear without errors	Passed

10. OUTPUT

Paste the actual terminal screenshots in the spaces below. A single screenshot may be used when it clearly shows multiple successful test cases.

Output 1: Function execution – greetStudent() test cases

```
function greetStudent(name) {  
    return "Hello, " + name + "!";  
}  
  
// Calling the function  
console.log("FUNCTION EXAMPLE");  
console.log(greetStudent("Venu"));  
console.log(greetStudent("Rahul"));
```

```
PS F:\full-stack-web-development\01-Observation\Task6-PBLpart2\5_jsOOP  
S> node function_classes.js  
FUNCTION EXAMPLE  
Hello, Venu!  
Hello, Rahul!
```

Output 2: Class objects – student1, student2 and student3 details

```
class Student {  
    // Constructor initializes the properties  
    constructor(name, rollNumber, branch) {  
        this.name = name;  
        this.rollNumber = rollNumber;  
        this.branch = branch;  
    }  
  
    // Method to display student details  
    displayDetails() {  
        console.log("Name:", this.name);  
        console.log("Roll Number:", this.rollNumber);  
        console.log("Branch:", this.branch);  
        console.log("-----");  
    }  
}
```

```
CLASS OBJECTS  
Name: Venu  
Roll Number: A24126552078  
Branch: CSM  
-----  
Name: Rahul  
Roll Number: A24126552079  
Branch: CSE  
-----  
Name: Priya  
Roll Number: A24126552080  
Branch: ECE  
-----
```

Output 3: Combined execution – multiple test cases passed in one terminal screenshot

```
PS F:\full-stack-web-development\01-Observation\Task6-PBLpart2\5_jsOOPS> node function_classes.js  
FUNCTION EXAMPLE  
Hello, Venu!  
Hello, Rahul!  
  
CLASS OBJECTS  
Name: Venu  
Roll Number: A24126552078  
Branch: CSM  
-----  
Name: Rahul  
Roll Number: A24126552079  
Branch: CSE  
-----  
Name: Priya  
Roll Number: A24126552080  
Branch: ECE  
-----
```

11. RESULT

Thus, JavaScript functions and classes were studied and demonstrated. A reusable function was used to perform a greeting task, and a Student class was created with common properties and a displayDetails() method. Multiple Student objects were successfully created from the same class with different property values.

12. SUBMISSION CONTENT

- JavaScript source file containing the complete program.
- Terminal output screenshots showing successful test-case execution.
- Completed laboratory observation / record.
- Final observation record converted to PDF, if required by the faculty.
- Source code and record maintained in the GitHub repository.